Karmada统一多集群架构介绍

1. 背景

1.1 为什么需要多集群

Kubernetes 从 1.8 版本起就声称单集群最多可支持5000 个节点和 15 万个Pod，实际上应该很少有公司会部署如此庞大的一个单集群，很多情况下因为各种各样的原因我们可能会部署多个集群，但是又想将它们统一起来管理，这时候就需要用到多集群。

多集群典型要求：

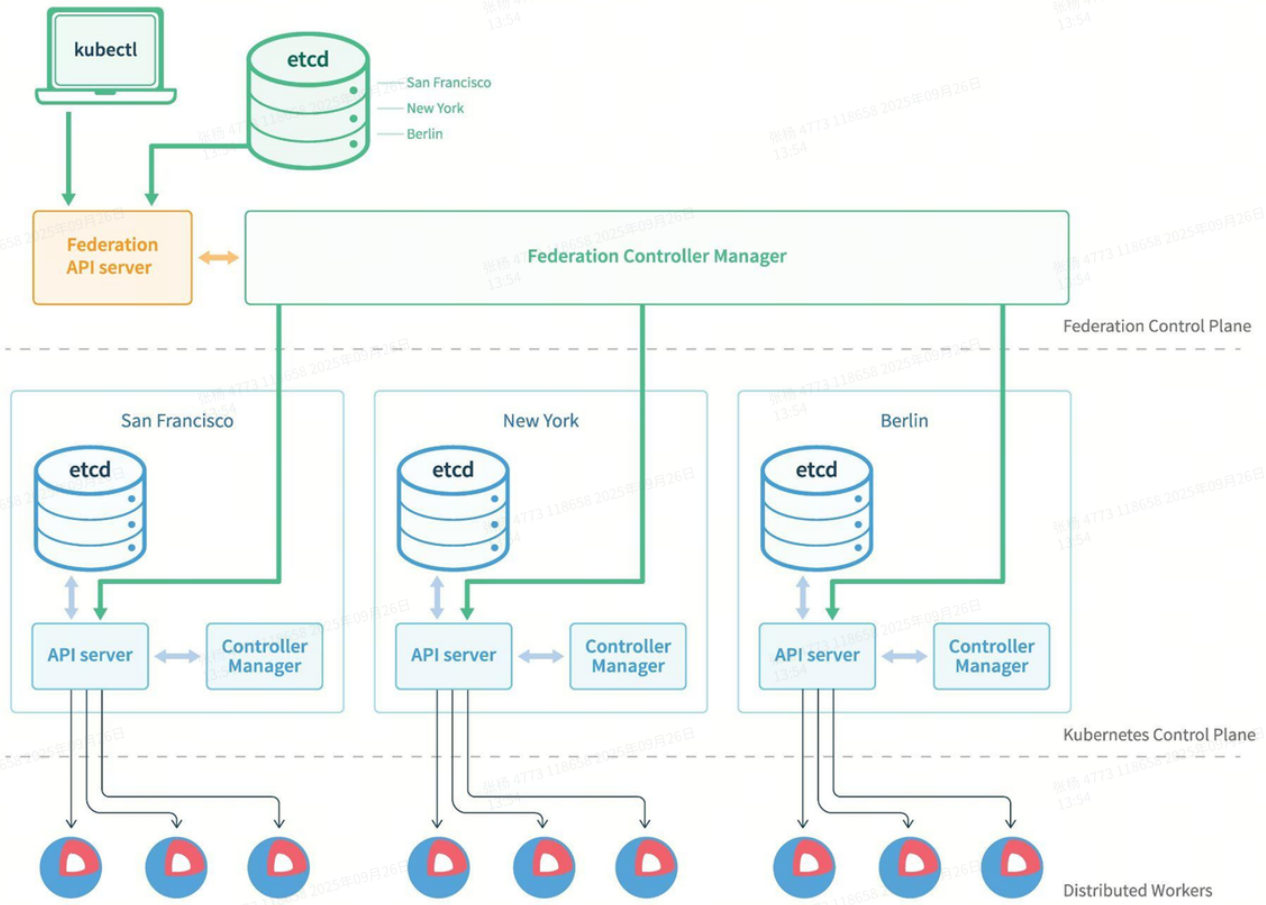
- 高可用：在多个集群上部署应用，可以最大限度地减少集群故障带来的影响
- 故障隔离：拥有多个小集群可能比单个大集群更利于故障隔离
- 地域分布：跨地域容灾
- 资源利用：弹性扩展、资源冗余、负载均衡
- 灾难恢复：单集群故障快速切换

1.2 希望多集群做什么

- 跨云跨集群统一接入统一管理
- 兼容K8s API
- 统一监控和运维能力
- 资源统一调度
- 应用数据协同一键迁移

1.3 Kubernetes Federation V1

最早的多集群项目，由 K8s 社区提出和维护。Federation v1 在 K8s v1.3左右社区就已经着手设计（Design Proposal），并在后面几个版本中发布了相关的组件与命令行工具（kubefed），用于帮助使用者快速建立联邦集群，并在 v1.6 时，进入了 Beta 阶段，但 Federation v1 在进入 Beta 后，就没有更进一步的发展，由于灵活性和 API 成熟度的问题，在 K8s v1.11 左右正式被弃用。



1.3.1 核心组件

- Federation API Server：类似 K8s API Server，对外提供统一的资源管理入口，但只允许使用 Adapter 拓展支持的 K8s 资源
- Controller Manager：提供多个集群间资源调度及状态同步，类似 kube-controller-manager
- Etcd：储存 Federation 的资源

1.3.2 局限性

- 只能同步少数内置资源：当时只支持 Deployment、DaemonSet、ConfigMap、Secret、Service 等少量资源类型，要想同步新资源就需要手动扩展 CRD。
- 调度策略单一，功能有限：没有灵活的放置（Placement）策略；大多数场景只是“全量”同步或基于“简单列表”同步。
- 健康检查与容灾不足：缺乏完善的成员集群健康检测、故障自动切换能力；如果某个集群宕机，用户需要手动干预。
- 维护成本高：联邦控制面本身是一套相对独立的组件集合，需要额外运维；社区活跃度有限，功能迭代缓慢。
- 使用体验欠佳：CRD 与原生资源差异较大，学习门槛高；调试与排错相对繁琐。

1.3.3 最终命运

- 在 Production 上较少采用，社区对其反馈“功能太有限、API 设计不够灵活”；
- 随着更先进的多集群方案逐渐出现（包括 Federation v2、KubeFed v2 以及后来的 Karmada 等），Federation v1 最终被官方标记为过时，并逐步被移除。

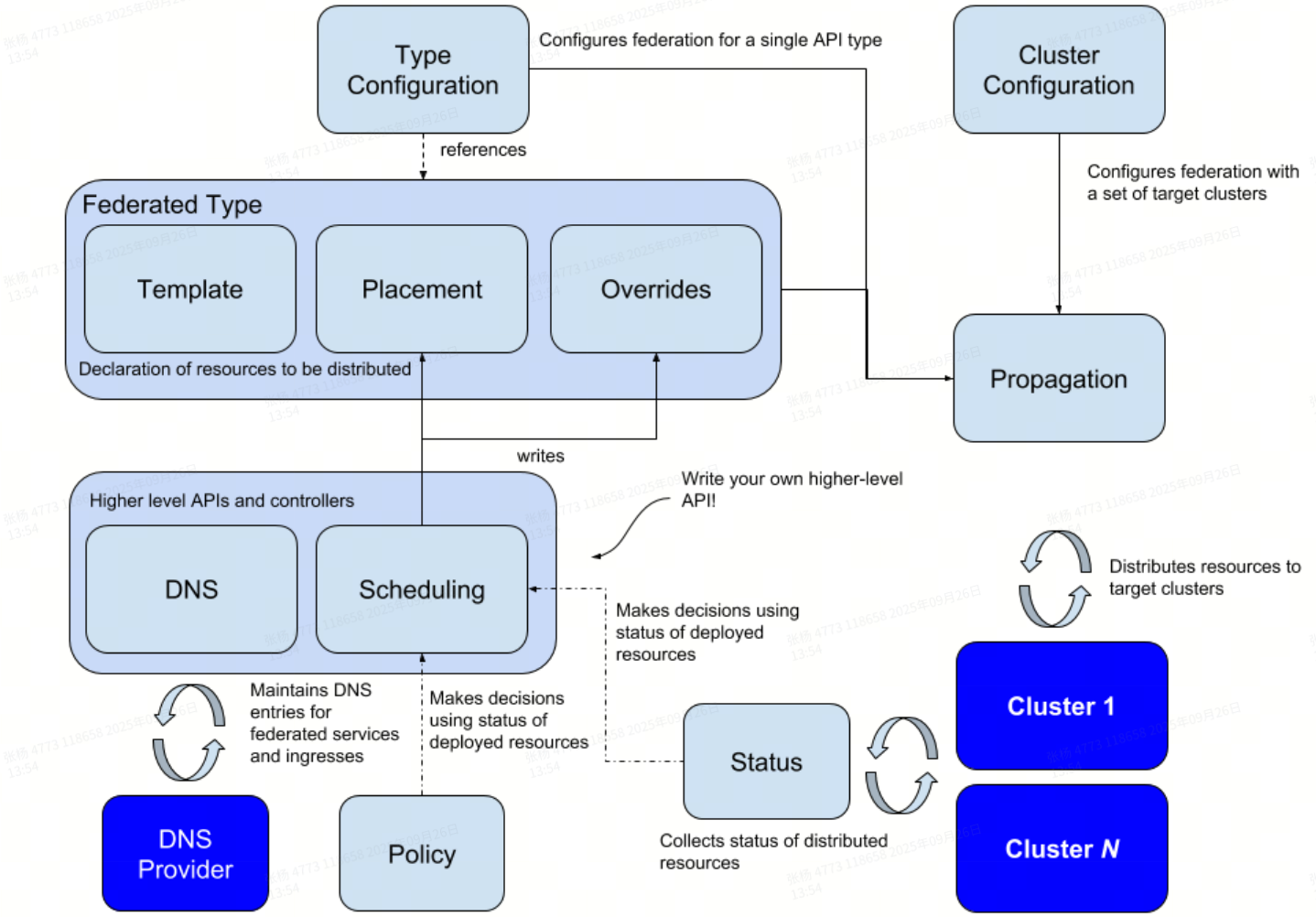
1.4 Kubernetes Federation V2

诞生时间：2018 年左右，Kubernetes 社区启动 Federation v2（又称 KubeFed v2），目标是“打造一个比 v1 更灵活、更易扩展的联邦框架”。

核心改进：

- CRD+Controller 模式：不通过单独的 API Server，而是采用原生 CRD（例如 FederatedDeployment, FederatedService, FederatedConfigMap）加上对应的 Controller，与 Kubernetes API Server 直接集成在一起。
- 扩展性更好：用户可以定义更多的联邦资源，也可以为自定义资源（CRD）创建对应的联邦控制器，让无限种资源类型都可跨集群同步。
- 调度策略增强：引入了 Placement、PropagationPolicy 等更灵活的资源同步与放置策略。
- 注重可插拔：调度与同步逻辑通过 Controller 实现，理论上可以替换或自定义。
- 更贴近原生 Kubernetes：联邦资源与原生资源在结构上保持更高一致性，运维体验更好。

1.4.1 核心概念



主要由两个组件构成：

- admission-webhook：提供了准入控制。
- FederatedTypeConfig Controller：注册 “要联邦的资源类型”，为这些类型自动创建对应的 CRD（如 Federated<Kind>）与 Controller。
- Sync Controller：监听用户创建的联邦资源（FederatedX），根据 Placement 策略将原生资源（Deployment、Service 等）同步到各集合群。
- Placement Controller：负责解析用户定义的 Placement（或称 “Override”、“Propagation”）策略，将同步放置到不同集群。

联邦 CRD 体系：

- FederatedTypeConfig：用于注册 “我想把这种类型的资源放到联邦里”，一旦注册，系统自动生成对应的联邦 CRD。
- FederatedDeployment、FederatedService、FederatedConfigMap 等：实际用户创建的对象，用于定义具体某个资源如何跨集群分发。
- Placement、Override 等：协助定义目标集群、字段覆盖、同步模式等。

1.4.2 优点与不足

优点：

- 扩展性增强：理论上可以联邦任何支持的 Kubernetes 资源，包括自定义 CRD。
- 与原生 Kubernetes 更贴合：无需运行额外的 API Server，直接用 kube‑api-server 接受请求。
- 多策略支持：内置 Placement、Override、Propagation 等策略，能满足常见跨集群同步需求。

不足：

- 社区维护活跃度下降：虽然最初迭代速度较快，但到 2020 年后社区关注度开始下降，提交与更新频率降低。
- 功能与性能不够"轻量"：对很多小型或中型场景，KubeFed 的学习与使用成本仍然偏高，需要了解较多 CRD、Controller 协作逻辑。
- 健康检测与容灾不够完善：虽然提供了健康检查机制，但自动故障切换、动态扩缩容等功能的成熟度不如后续项目。
- 缺少统一的“轻量调度层”：其调度逻辑分散在多个 Controller 中，定制化需要深入源码或自行扩展，门槛较高。

1.4.3 最终定位

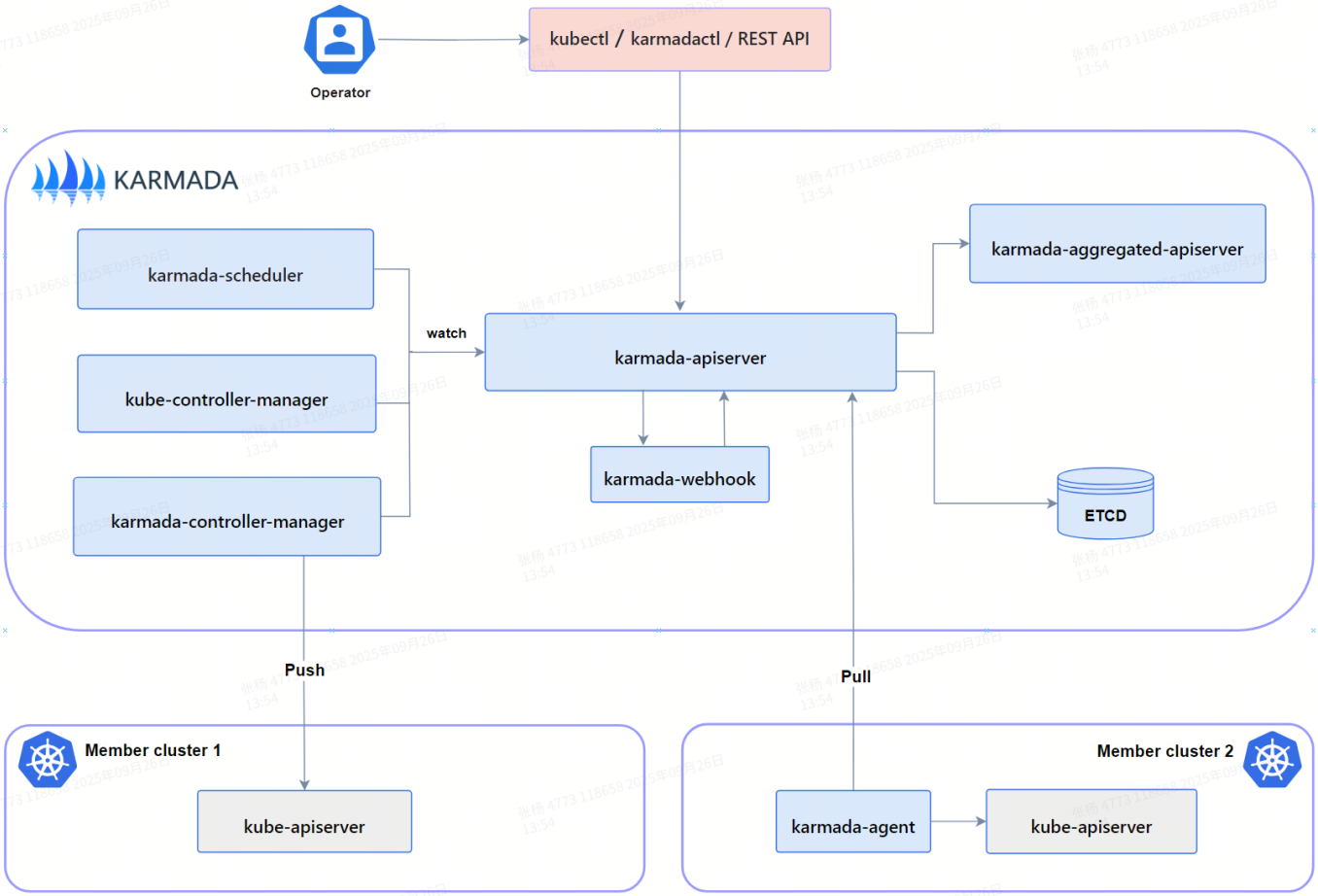
虽然 Federation v2 在一段时间内被很多大型企业或跨国团队用于多集群场景，但随着社区资源与关注度下降、维护成本升高，其生态逐渐衰退。同时，出现了如 Rancher Fleet、Argo CD ApplicationSet、Kubefed3（社区 fork）等多集群/多仓库同步方案，让 Federation 的“联邦”概念逐渐分散。直到 Karmada（以及类似 GitOps 方向的 Fleet/Argo CD）出现，才让多集群编排重新成为社区热点。

2. Karmada概览与架构

Karmada（Kubernetes Armada）是一个开源的多集群管理平台，旨在提供一种高效、可靠的方法来管理和操作多个Kubernetes集群。它允许用户通过单一的控制面板对跨云、跨区域的Kubernetes集群进行统一的操作和资源管理，无需修改现有的Kubernetes应用。

karmada在kubernetes Federation v1/v2基础开发，其概念继承至这个版本。

2.1 核心组件



一个完整且可工作的 Karmada 控制平面由以下组件组成。 `karmada-agent` 可以是可选的， 这取决于集群注册模式。

karmada-apiserver

API 服务器是 Karmada 控制平面的一个组件，对外暴露 Karmada API 以及 Kubernetes 原生API，API 服务器是 Karmada 控制平面的前端。

Karmada API 服务器是直接使用 Kubernetes 的 `kube-apiserver` 实现的，因此 Karmada 与 Kubernetes API 自然兼容。 这也使得 Karmada 更容易实现与 Kubernetes 生态系统的集成，例如允许用户使用 `kubectl` 来操作 Karmada、 与 ArgoCD 集成、与 Flux 集成等等。

karmada-aggregated-apiserver

聚合 API 服务器是使用 Kubernetes API 聚合层技术实现的扩展 API 服务器。 它提供了Cluster API以及相应的子资源， 例如 `cluster/status` 和 `cluster/proxy` ，实现了聚合 Kubernetes API Endpoint 等可以通过 karmada-apiserver 访问成员集群的高级功能。

kube-controller-manager

kube-controller-manager 由一组控制器组成，Karmada 只是从 Kubernetes 的官方版本中挑选了一些控制器，以保持与原生控制器一致的用户体验和行为。

值得注意的是，并非所有的原生控制器都是 Karmada 所需要的， 推荐的控制器请参阅 推荐的控制器。

> 注意：当用户向 Karmada API 服务器提交 Deployment 或其他 Kubernetes 标准资源时，它们只记录在 Karmada 控制平面的 etcd 中。 随后，这些资源会向成员集群同步。然而，这些部署资源不会在 Karmada 控制平面集群中进行 reconcile 过程（例如创建Pod）。

karmada-controller-manager

Karmada 控制器管理器运行了各种自定义控制器进程。

控制器负责监视Karmada对象，并与底层集群的API服务器通信，以创建原生的 Kubernetes 资源。

所有的控制器列举在 Karmada 控制器。

karmada-scheduler

karmada-scheduler 负责将 Kubernetes 原生API资源对象（以及CRD资源）调度到成员集群。

调度器依据策略约束和可用资源来确定哪些集群对调度队列中的资源是可用的，然后调度器对每个可用集群进行打分排序，并将资源绑定到最合适的集群。

karmada-webhook

karmada-webhook 是用于接收 karmada/Kubernetes API 请求的 HTTP 回调，并对请求进行处理。 你可以定义两种类型的 karmada-webhook，即验证性质的 webhook 和修改性质的 webhook。 修改性质的准入 webhook 会先被调用。它们可以更改发送到 Karmada API 服务器的对象以执行自定义的设置默认值操作。

在完成了所有对象修改并且 Karmada API 服务器也验证了所传入的对象之后，验证性质的 webhook 会被调用，并通过拒绝请求的方式来强制实施自定义的策略。

> 说明：如果 Webhook 需要保证它们所看到的是对象的最终状态以实施某种策略。则应使用验证性质的 webhook，因为对象被修改性质 webhook 看到之后仍然可能被修改。

etcd

一致且高可用的键值存储，用作 Karmada 的所有 Karmada/Kubernetes 资源对象数据的后台数据库。

如果你的 Karmada 使用 etcd 作为其后台数据库，请确保你针对这些数据有一份备份计划。

你可以在官方文档中找到有关 etcd 的深入知识。

karmada-agent

Karmada 有 Push 和 Pull 两种集群注册模式，karmada-agent 应部署在每个 Pull 模式的成员集群上。 它可以将特定集群注册到 Karmada 控制平面，并将工作负载清单从 Karmada 控制平面同步到成员集群。 此外，它也负责将成员集群及其资源的状态同步到 Karmada 控制平面。

集群注册模式

karmada对工作集群管理有两种方式：

1. Push模式

karmada直接访问工作集群的控制面，完成集群注册

2. Pull模式

工作集群部署agent组件，工作集群反向连接karmada，完成注册

2.2 关键资源

Resource Template资源模板

Karmada 使用 Kubernetes 原生 API 定义联邦资源模板，以便轻松与现有 Kubernetes 采用的工具进行集成。

Propagation Policy传播策略

Karmada 提供了一个独立的 Propagation(placement) Policy API 来定义多集群的调度要求。

- 支持 1:N 的策略映射机制。用户无需每次创建联邦应用时都标明调度约束。

Override Policy差异化策略

Karmada 为不同的集群提供了一个可自动化生产独立配置的 Override Policy API。例如：

- 基于成员集群所在区域自动配置不同镜像仓库地址。
- 根据集群不同的云厂商，可以使用不同的存储类。

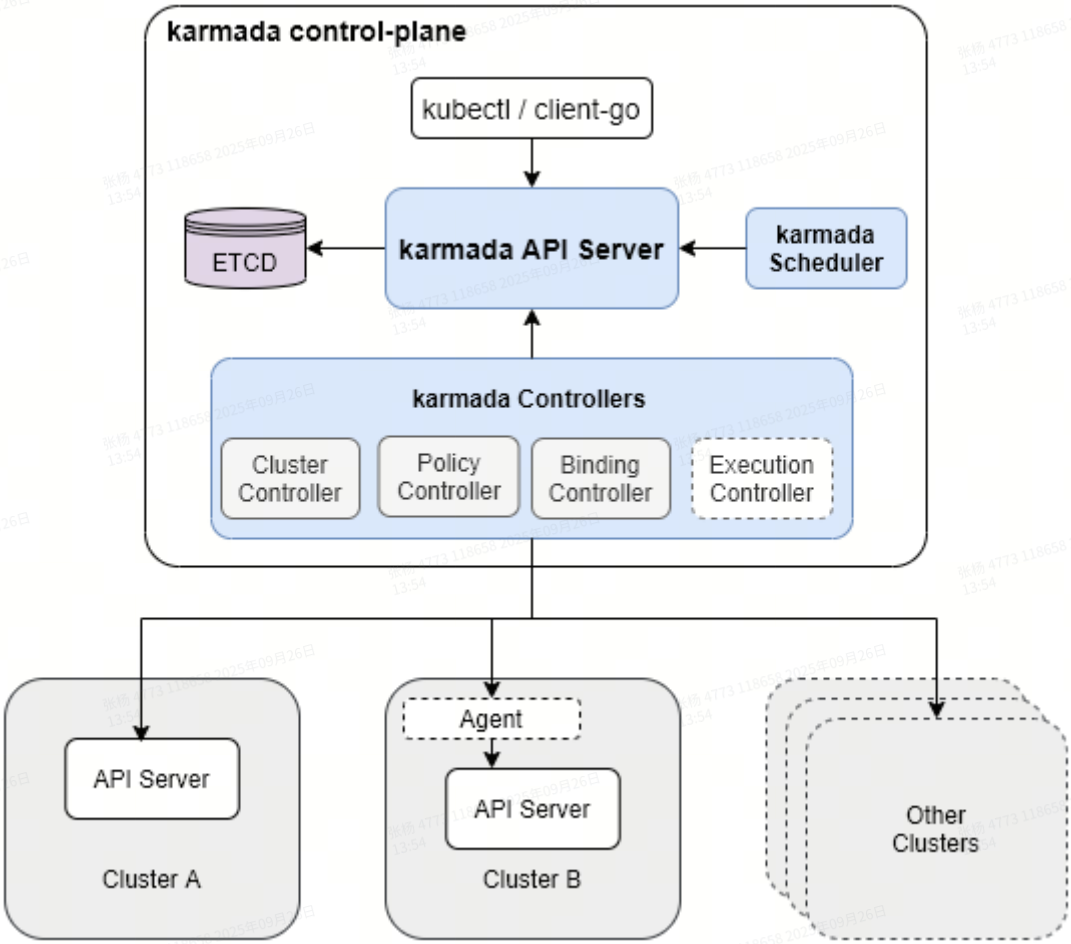
ResourceBinding

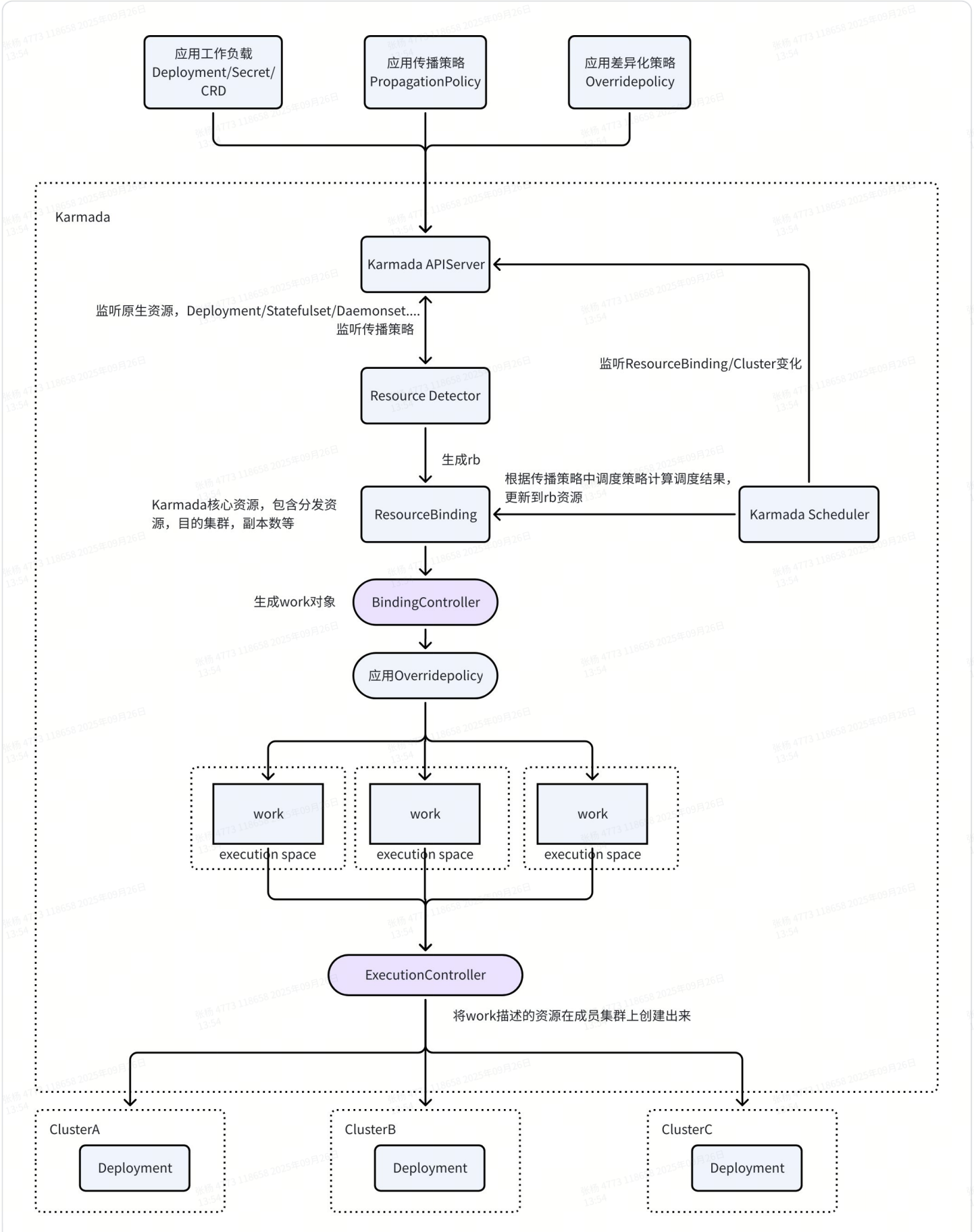
关键CRD（自定义资源），主要用于记录资源模板与目标集群的绑定关系。包含资源定义，策略定义，副本资源需求，目的集群等信息。

Work

Karmada 中的 Work 资源 是多云资源分发的核心执行单元，用于将资源模板精确同步到目标成员集群。

2.3 整体架构


karmada control-plane
kubectl / client-go
ETCD
karmada API Server
karmada Scheduler
karmada Controllers
Cluster Controller
Policy Controller
Binding Controller
Execution Controller
API Server
Cluster A
Agent
API Server
Cluster B
Other Clusters



karmada拥有自己完整控制面，对外提供标准的k8s原生API，呈现出完整k8s集群集合，使一个k8s工作负载在多个工作集群运行而不需要更改应用程序及配置。

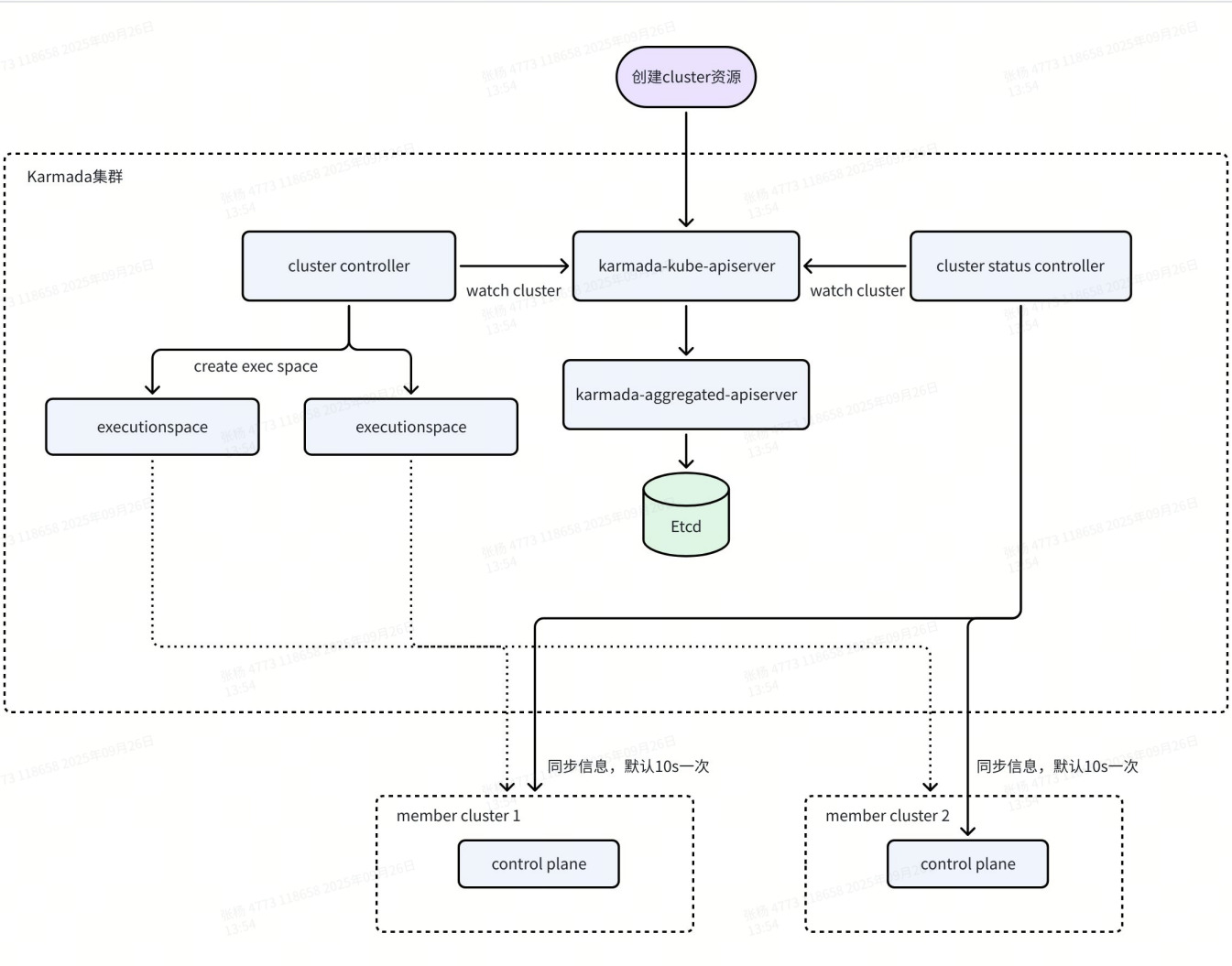
3. 核心资源

3.1 集群注册

Cluster资源定义：

代码块

```yaml
apiVersion: cluster.karmada.io/v1alpha1
kind: Cluster
metadata:
  name: member-cluster-1 # 给您的集群起的名字
spec:
  syncMode: Push
  secretRef:
    name: member-cluster1-secret # 引用之前创建的Secret的名字
    namespace: karmada-system
  apiEndpoint: "https://10.195.72.98:6443" # 成员集群API Server的地址和端口
```



3.2 传播策略

传播策略有两种PropagationPolicy和ClusterPropagationPolicy

PropagationPolicy 主要用于针对特定命名空间内的资源进行细粒度的控制，适合于那些需要对每个命名空间单独配置的情况。

ClusterPropagationPolicy 提供了一种更广泛的方法来管理跨所有命名空间的资源传播，简化了全局性的资源配置，减少了重复的工作量。

3.2.1 指定资源

传播策略指定资源，可以和资源名称和命名空间限制

代码块

```yaml
# propagationpolicy.yaml
apiVersion: policy.karmada.io/v1alpha1
kind: PropagationPolicy
metadata:
  name: example-policy # The default namespace is `default`.
spec:
  resourceSelectors:
    - apiVersion: apps/v1
      kind: Deployment
      name: nginx # If no namespace is specified, the namespace is inherited from the parent object scope.
  placement:
    clusterAffinity:
      clusterNames:
        - member1
```

也可以只指定命名空间，命名空间下的工作负载执行当前传播策略：

代码块

```yaml
# propagationpolicy.yaml
apiVersion: policy.karmada.io/v1alpha1
kind: PropagationPolicy
metadata:
  name: example-policy # The default namespace is `default`.
spec:
  resourceSelectors:
    - apiVersion: apps/v1
      kind: Deployment
      Namespace: test
  placement:
    clusterAffinity:
      clusterNames:
        - member1
```

3.2.2 指定集群

传播策略集群选择方式，使用ClusterNames字段，可以指定候选集群名称，以PropagationPolicy举例

代码块

```
apiVersion: policy.karmada.io/v1alpha1
kind: PropagationPolicy
metadata:
  name: nginx-propagation
spec:
  #...
  placement:
    clusterAffinity:
      clusterNames:
        - member1
        - member2
    #...
```

也可以使用FieldSelector，指定特定字段可以是可用区也可以是Region

代码块

```
apiVersion: policy.karmada.io/v1alpha1
kind: PropagationPolicy
metadata:
  name: nginx-propagation
spec:
  #...
  placement:
    clusterAffinity:
      fieldSelector:
        matchExpressions:
        - key: region
          operator: In
          values:
          - banan
```

3.2.3 指定集群组

cluster affinity groups使用，策略可以声明多个集群组，调度器按顺序评估集群组，如果某组不满足调度要求，则该组下的集群会被忽略，选择下一组集群。

重要提示：

- ClusterAffinities与ClusterAffinity不能同时存在。

- 如果既未设置ClusterAffinity也未设置ClusterAffinities，任何集群都是调度的候选对象。

这样配置会优先调用local-clusters这组：

代码块

```yaml
apiVersion: policy.karmada.io/v1alpha1
kind: PropagationPolicy
metadata:
  name: test-propagation
spec:
  #...
  placement:
    clusterAffinities:
      - affinityName: local-clusters
        clusterNames:
          - local-member1
          - local-member2
      - affinityName: cloud-clusters
        clusterNames:
          - public-cloud-member1
          - public-cloud-member2
    #...
```

主备集群分组，优先调度到primary-clusters集群组：

代码块

```yaml
apiVersion: policy.karmada.io/v1alpha1
kind: PropagationPolicy
metadata:
  name: test-propagation
spec:
  #...
  placement:
    clusterAffinities:
      - affinityName: primary-clusters
        clusterNames:
          - member1
      - affinityName: backup-clusters
        clusterNames:
          - member1
          - member2
    #...
```

集群可以设置污点，传播策略配置容忍。配置cluster污点：

代码块

```
karmadactl taint clusters foo dedicated=special-user:NoSchedule

```

配置传播策略容忍度：

代码块

```yaml
apiVersion: policy.karmada.io/v1alpha1
kind: PropagationPolicy
metadata:
  name: nginx-propagation
spec:
  #...
  placement:
    clusterTolerations:
    - key: dedicated
      value: special-user
      Effect: NoSchedule
```

3.2.4 副本调度策略

`replicaScheduling` 包含两种策略Duplicated(全量)和Divided(拆分)。

3.2.4.1 全量策略Duplicated

全量策略会把工作负载所有副本分发到所有指定的成员集群里，比如Deploy有2个副本，Duplicated会把2个副本分配到所有候选集群里，被调度到的集群会运行2个副本。适用全量多活部署。

代码块

```yaml
apiVersion: policy.karmada.io/v1alpha1
kind: PropagationPolicy
metadata:
  name: nginx-propagation
spec:
  #...
  placement:
    replicaScheduling:
     replicaSchedulingType: Duplicated
    spreadConstraints:
      - spreadByField: region
        maxGroups: 2
        minGroups: 2
      - spreadByField: cluster
        maxGroups: 1
```

```
16          minGroups: 1
```

3.2.4.2 拆分策略Divided

`Divided` 策略包含 `Weighted` 和 `Aggregated` 两种类型。

3.2.4.3 集群资源模型

通用集群资源模型

代码块

```
1   resourceSummary:
2       allocatable:
3         cpu: "4"
4         ephemeral-storage: 206291924Ki
5         hugepages-1Gi: "0"
6         hugepages-2Mi: "0"
7         memory: 16265856Ki
8         pods: "110"
9       allocated:
10        cpu: 950m
11        memory: 290Mi
12        pods: "11"
```

`ResourceSummary` 描述了集群的整体可用资源。 但是， `ResourceSummary` 不够精确，它机械地统计所有节点上的资源，而忽略了节点上的碎片资源。例如，一个有 2000 个节点的集群，每个节点上只剩下1核CPU。 从 `ResourceSummary` 中我们获知集群还有 2000核CPU，但事实上，这个集群甚至无法运行任何需要1核CPU以上的Pod实例。

自定义集群资源模型

代码块

```
1   resourceModels:
2     - grade: 0
3       ranges:
4       - max: "1"
5         min: "0"
6         name: cpu
7       - max: 4Gi
8         min: "0"
9         name: memory
10    - grade: 1
11      ranges:
12      - max: "2"
13        min: "1"
```

```
      name: cpu
    - max: 16Gi
      min: 4Gi
      name: memory
  - grade: 2
    ranges:
    - max: "4"
      min: "2"
      name: cpu
    - max: 32Gi
      min: 16Gi
      name: memory
  - grade: 3
    ranges:
    - max: "8"
      min: "4"
      name: cpu
    - max: 64Gi
      min: 32Gi
      name: memory
  - grade: 4
    ranges:
    - max: "16"
      min: "8"
      name: cpu
    - max: 128Gi
      min: 64Gi
      name: memory
  - grade: 5
    ranges:
    - max: "32"
      min: "16"
      name: cpu
    - max: 256Gi
      min: 128Gi
      name: memory
  - grade: 6
    ranges:
    - max: "64"
      min: "32"
      name: cpu
    - max: 512Gi
      min: 256Gi
      name: memory
  - grade: 7
    ranges:
    - max: "128"
```

```
        min: "64"
        name: cpu
      - max: 1Ti
        min: 512Gi
        name: memory
    - grade: 8
      ranges:
      - max: "9223372036854775807"
        min: "128"
        name: cpu
      - max: "9223372036854775807"
        min: 1Ti
        name: memory
```

`自定义集群资源模型` 将节点划分为不同区间的等级，并且当一个Pod实例需要调度到特定集群时，`karmada-scheduler` 根据将要调度的实例资源请求比较不同集群中满足要求的节点数，并将实例调度到满足要求的节点数更多的集群。 假设有三个注册在Karmada控制面的成员集群，采用默认设置的集群资源模型，这些集群的剩余可用资源情况如下。

成员集群1:

代码块

```
spec:
...
  - grade: 2
    ranges:
    - max: "4"
      min: "2"
      name: cpu
    - max: 32Gi
      min: 16Gi
      name: memory
  - grade: 3
    ranges:
    - max: "8"
      min: "4"
      name: cpu
    - max: 64Gi
      min: 32Gi
      name: memory
...
...
status:
  - count: 1
    grade: 2
```

```
    - count: 6
      grade: 3
```

成员集群2:

代码块

```
spec:
...
    - grade: 2
      ranges:
      - max: "4"
        min: "2"
        name: cpu
      - max: 32Gi
        min: 16Gi
        name: memory
    - grade: 3
      ranges:
      - max: "8"
        min: "4"
        name: cpu
      - max: 64Gi
        min: 32Gi
        name: memory
...
...
status:
    - count: 4
      grade: 2
    - count: 4
      grade: 3
```

成员集群3:

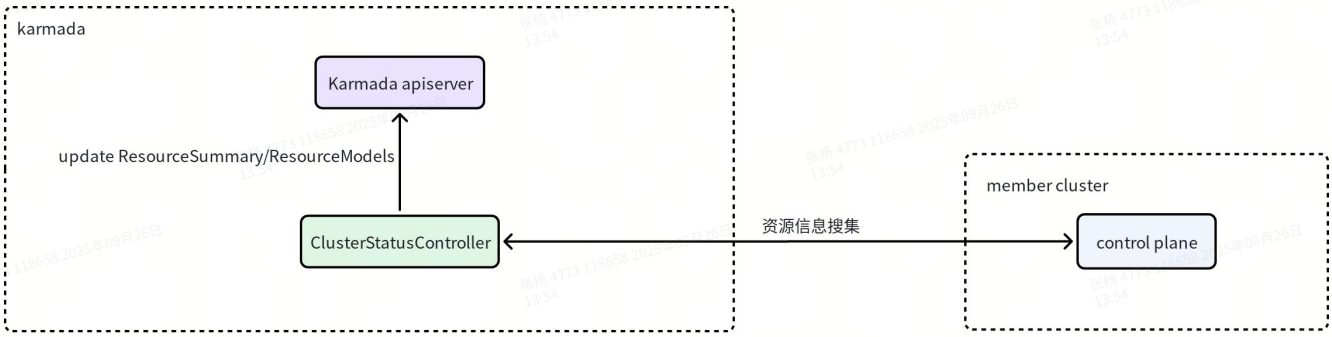
代码块

```
spec:
...
    - grade: 6
      ranges:
      - max: "64"
        min: "32"
        name: cpu
      - max: 512Gi
        min: 256Gi
```

```
      name: memory
...
...
status:
  - count: 1
    grade: 6
```

假设这时过来一个Pod的调度请求，Pod的资源请求是3核CPU和20Gi内存。那么，Karmada认为所有满足等级2及以上的节点满足此要求。考虑到不同集群可用节点的数量，调度器更倾向于Pod调度到成员集群3。

Cluster	member1	member2	member3
AvailableReplicas	6 * min(4/3,32/20) = 6	4 * min(4/3,32/20) = 4	1*min(32/3,256/20) = 10



3.2.4.3.1 Weighted策略

Weighted权重包含静态权重 `StaticWeightList` 和动态权重 `DynamicWeight` 。静态权重先定义各个集群权重。调度器根据配置的权重大小按比例分配工作负载副本。

代码块

```
apiVersion: policy.karmada.io/v1alpha1
kind: PropagationPolicy
metadata:
  name: nginx-propagation
spec:
  #...
  placement:
    replicaScheduling:
      replicaDivisionPreference: Weighted
      replicaSchedulingType: Divided
      weightPreference:
        staticWeightList:
          - targetCluster:
```

```
      clusterNames:
        - member1
    weight: 1
  - targetCluster:
      clusterNames:
        - member2
    weight: 2
```

动态权重DynamicWeight，顾名思义scheduler会根据成员集群resource model计算出可用副本情况做分配。例如一个负载有12个副本，有3个候选集群A/B/C的可用副本为A:6,B:12,C:18。可用副本比例6:12:18->1:2:3。所以最后分配比例A:2，B:4，C:6。

代码块

```
apiVersion: policy.karmada.io/v1alpha1
kind: PropagationPolicy
metadata:
  name: nginx-propagation
spec:
  #...
  placement:
    replicaScheduling:
      replicaDivisionPreference: Weighted
      replicaSchedulingType: Divided
      weightPreference:
        dynamicWeight: AvailableReplicas
```

3.2.4.3.2 Aggregated策略

`Aggregated` 模式：在副本划分过程中，尽可能将副本集中到尽可能少的集群，同时在划分时考虑各集群的资源可用性，基于集群resourcemodel做评估。

代码块

```
apiVersion: policy.karmada.io/v1alpha1
kind: PropagationPolicy
metadata:
  name: nginx-propagation
spec:
  #...
  placement:
    replicaScheduling:
      replicaDivisionPreference: Aggregated
      replicaSchedulingType: Divided
```

3.3 差异化策略

差异化策略有两种OverridePolicy和ClusterOverridePolicy，覆盖策略类似于应用模版，在安装到集群替换应用字段比如镜像地址，命令行等等

Overidepolicy主要用于针对特定命名空间内的资源进行覆盖

代码块

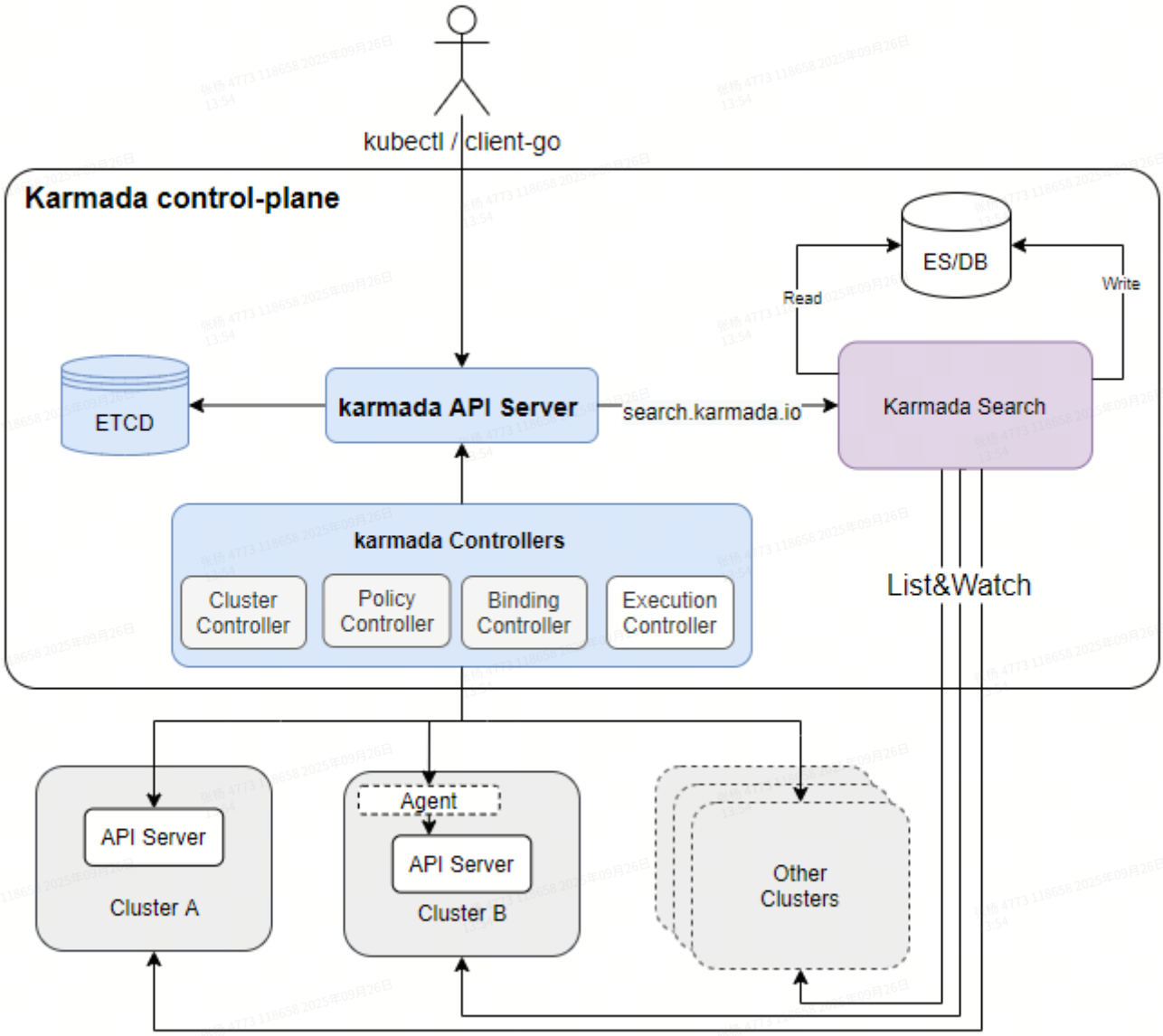
```yaml
apiVersion: policy.karmada.io/v1alpha1
kind: OverridePolicy
metadata:
  name: example
spec:
  resourceSelectors:
    - apiVersion: apps/v1
      kind: Deployment
      name: nginx
      namespace: test
      labelSelector:
        matchLabels:
          app: nginx
  overrideRules:
    - targetCluster:
        fieldSelector:
          matchExpressions:
            - key: zone
              operator: In
              values:
                - bn
      - overriders:
        imageOverrider:
          - component: Registry
            operator: replace
            value: test-repo
```

ClusterOveridepolicy全局性资源配置，跨命名空间资源覆盖

4. 主要功能

4.1 全局资源检索

Karmada Search实现多集群全局检索，它提供cache层和检索引擎能力查询搜索跨集群的资源类型。



ResourceRegistry资源例子

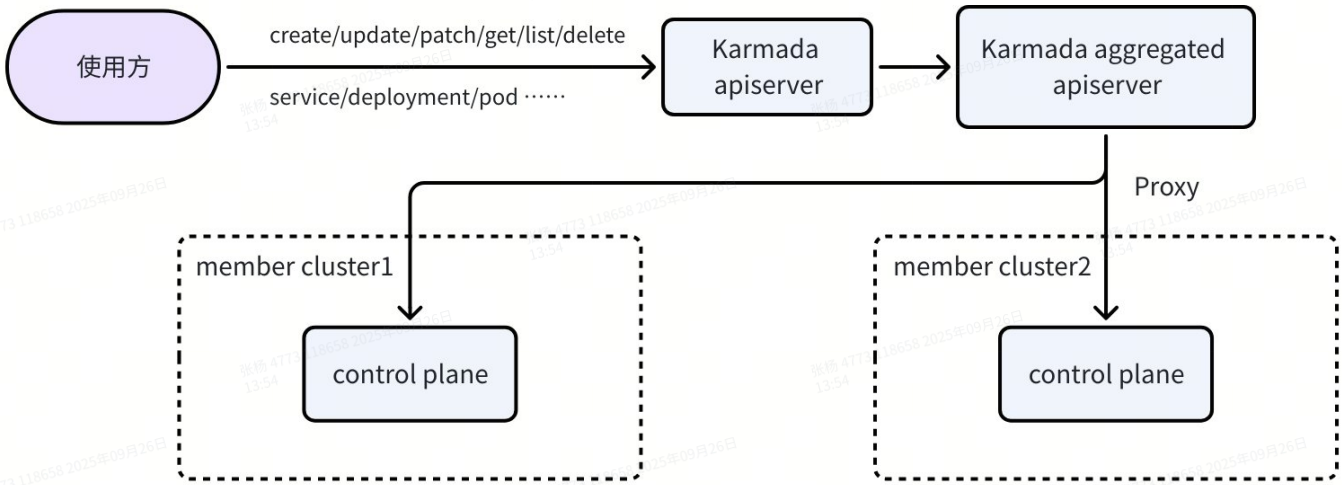
代码块

```yaml
### deployment-search.yaml
apiVersion: search.karmada.io/v1alpha1
kind: ResourceRegistry
metadata:
  name: deployment-search
spec:
  backendStore:
    openSearch:
      addresses:
        - http://karmada-opensearch.karmada-system.svc:9200
  targetCluster:
```

```
      clusterNames:
        - member1
        - member2
    resourceSelectors:
      - apiVersion: apps/v1
        kind: Deployment
```

4.2 统一资源视图

代理控制面对成员集群 API Server 的访问。允许使用方通过Karmada控制面来访问工作集群上的所有资源。



资源访问URI例子：

代码块

```
/apis/cluster.karmada.io/v1alpha1/clusters/{clusterName}/proxy/api/v1/namespaces/<ns>/pods/<pod-name>

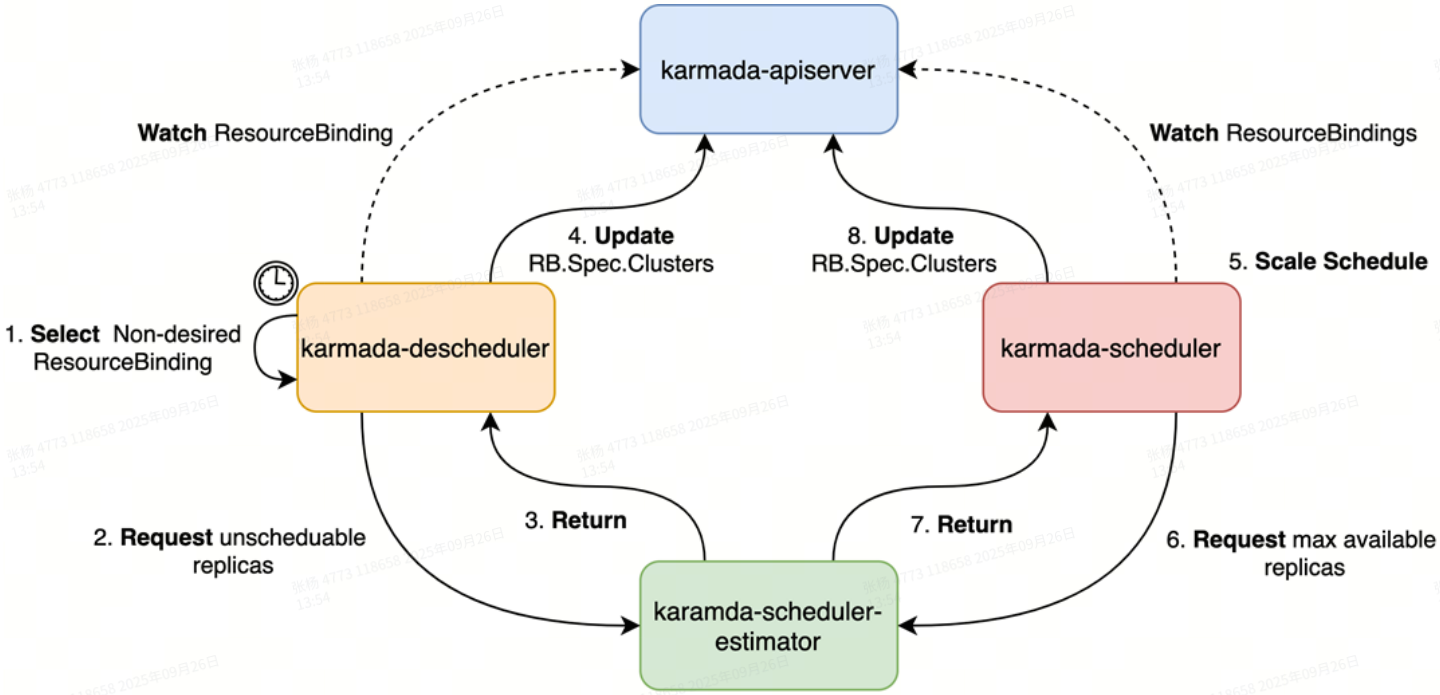
```

4.3 应用反调度

由于Karmada多集群环境具有高度动态性且状态随时间变化，当某集群资源不足时（例如集群节点故障导致无法容纳Pod，或估算器存在不可避免的估算偏差），可能需要将已运行的副本迁移至其他集群。

karmada-descheduler默认每2分钟检测一次所有部署。每个检测周期中，它会通过调用karmada-scheduler-estimator获取目标调度集群内无法调度的副本数量，随后从缩减的spec.clusters中驱逐这些副本，并触发karmada-scheduler根据当前状况执行"弹性调度"。需注意，该机制仅在副本调度策略为动态划分时生效。

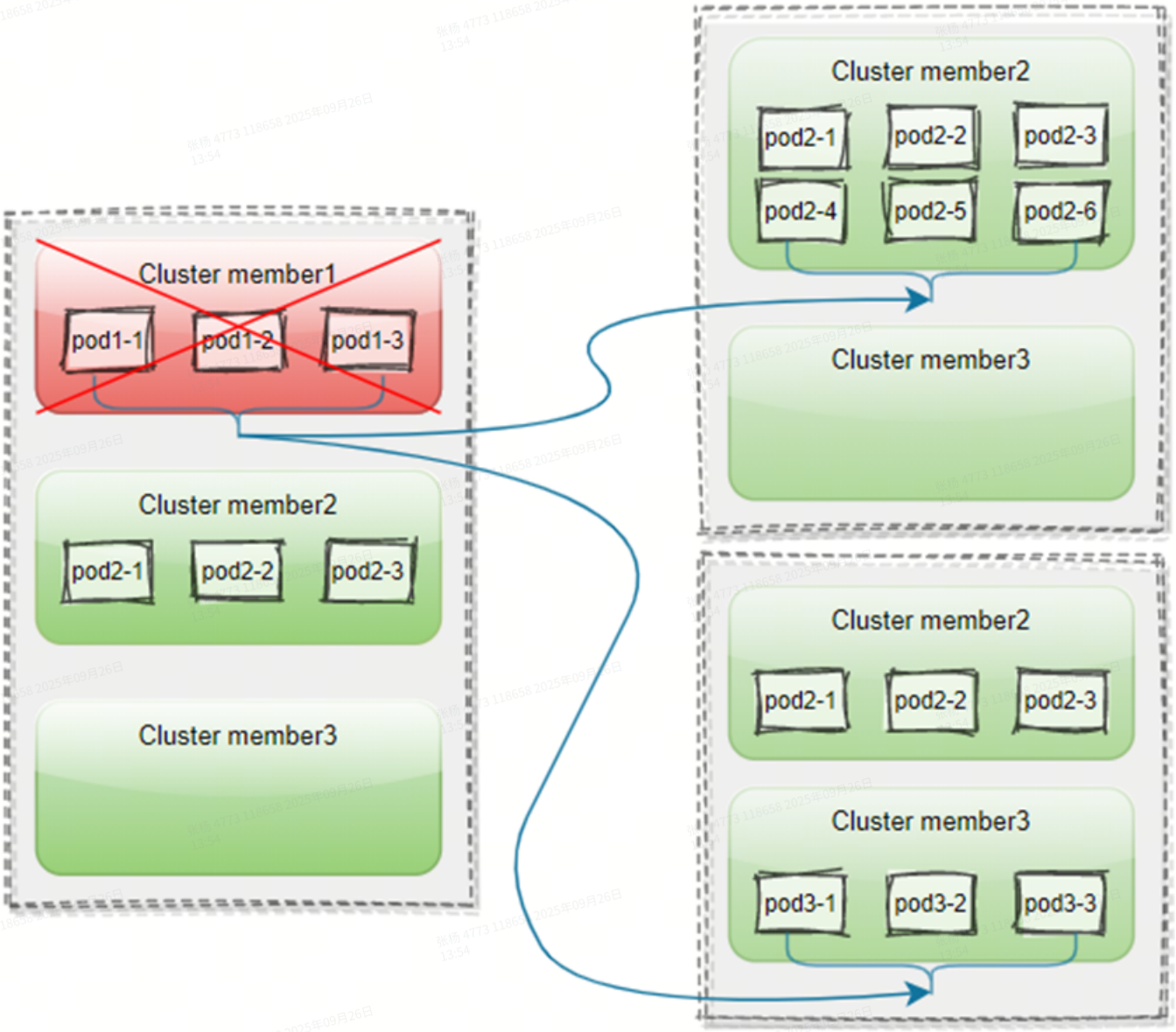
如果一个成员集群没有足够的资源容纳其中的 Pod，Karmada 会重新调度 Pod。整体的重调状态图如下：

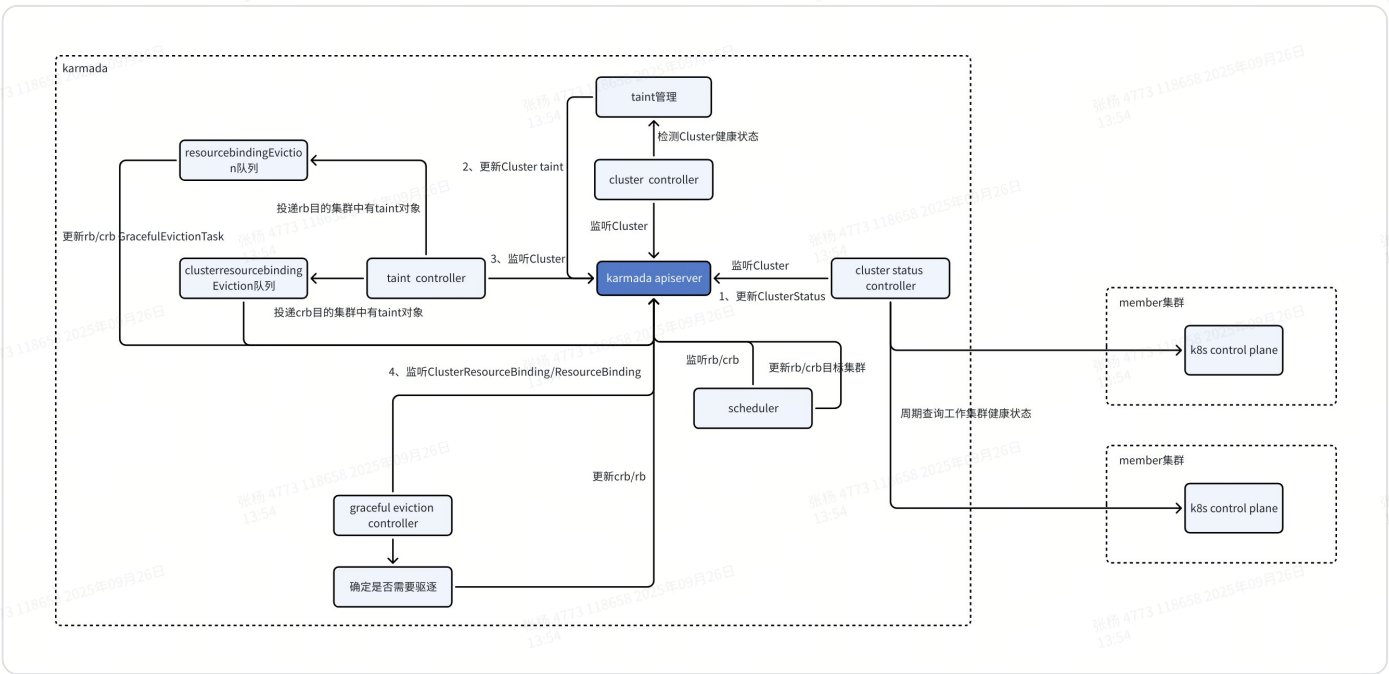


4.4 集群故障迁移

- 集群故障迁移：
  - Karmada 支持用户设置分发策略，在集群发生故障后，将故障集群实例进行自动的集中式或分散式的迁移。
- 集群污点设置：
  - 当用户为集群设置污点，且资源分发策略无法容忍污点时，Karmada 也会自动触发集群实例的迁移。
- 服务不断：
  - 在实例迁移过程中，Karmada 能够保证服务实例不跌零，从而确保服务不会断服。

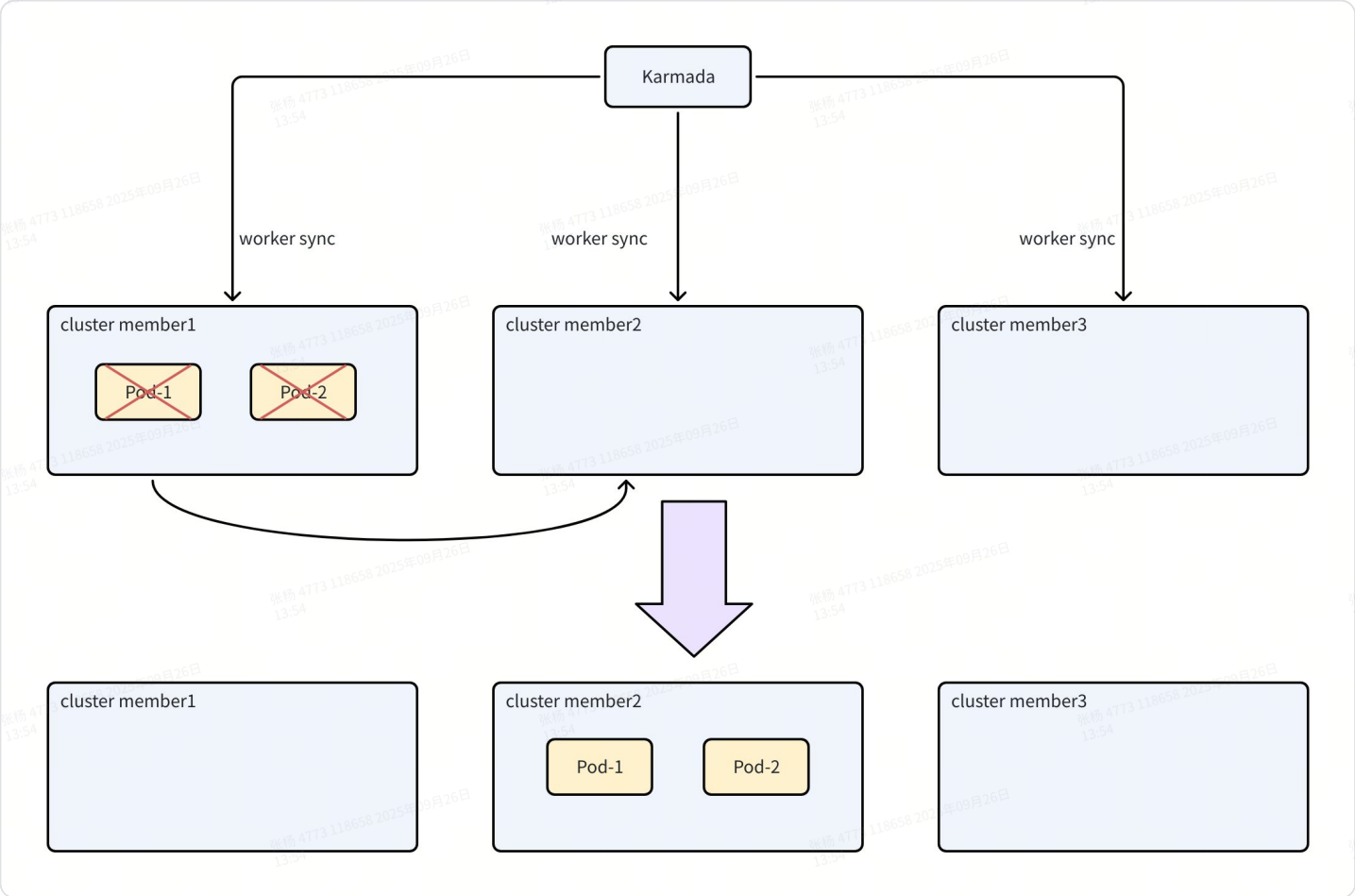
Karmada 支持成员集群的故障迁移，一个成员集群故障会导致集群实例的迁移，如下图所示：


Cluster member1
pod1-1
pod1-2
pod1-3
Cluster member2
pod2-1
pod2-2
pod2-3
Cluster member3
Cluster member2
pod2-1
pod2-2
pod2-3
pod2-4
pod2-5
pod2-6
Cluster member3
Cluster member2
pod2-1
pod2-2
pod2-3
Cluster member3
pod3-1
pod3-2
pod3-3


karmada
taint管理
检测Cluster健康状态
resourcebindingEvictio n队列
2、更新Cluster taint
cluster controller
投递rb目的集群中有taint对象
监听Cluster
更新rb/crb GracefulEvictionTask
clusterresourcebinding Eviction队列
taint controller
3、监听Cluster
karmada apiserver
监听Cluster
cluster status controller
1、更新ClusterStatus
投递crb目的集群中有taint对象
member集群
k8s control plane
监听rb/crb
更新rb/crb目标集群
4、监听ClusterResourceBinding/ResourceBinding
scheduler
周期查询工作集群健康状态
member集群
更新crb/rb
graceful eviction controller
k8s control plane
确定是否需要驱逐

4.5 应用故障迁移

Karmada可以从应用的角度提供一种故障迁移的手段。



配置应用故障迁移

PropagationPolicy 的 `.spec.failover.application` 字段可以来表示应用故障迁移的规则。

它有以下字段可以设置：

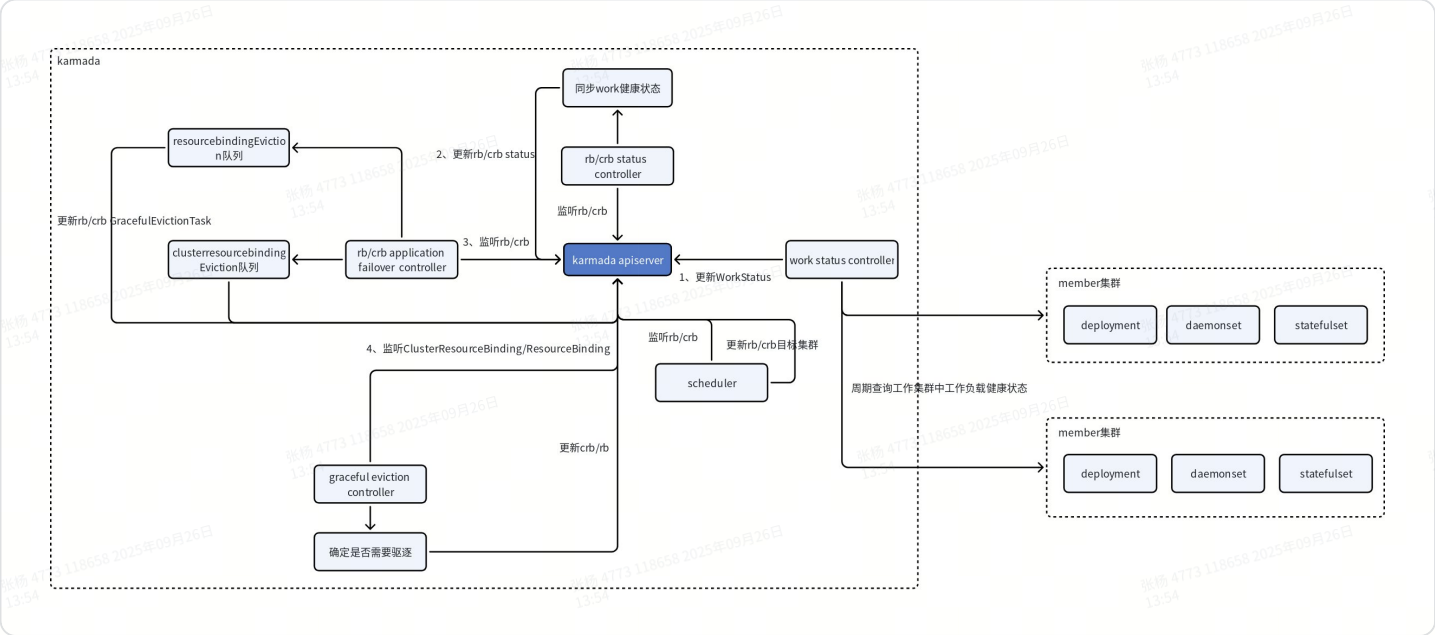
- DecisionConditions
- PurgeMode
- GracePeriodSeconds

`DecisionConditions` 代表执行故障迁移的决定条件。只有当其中的所有条件都满足了应用的故障迁移才会执行。 目前，它包括对应用不健康状态的容忍时间，默认为 300s。

配置驱逐模式

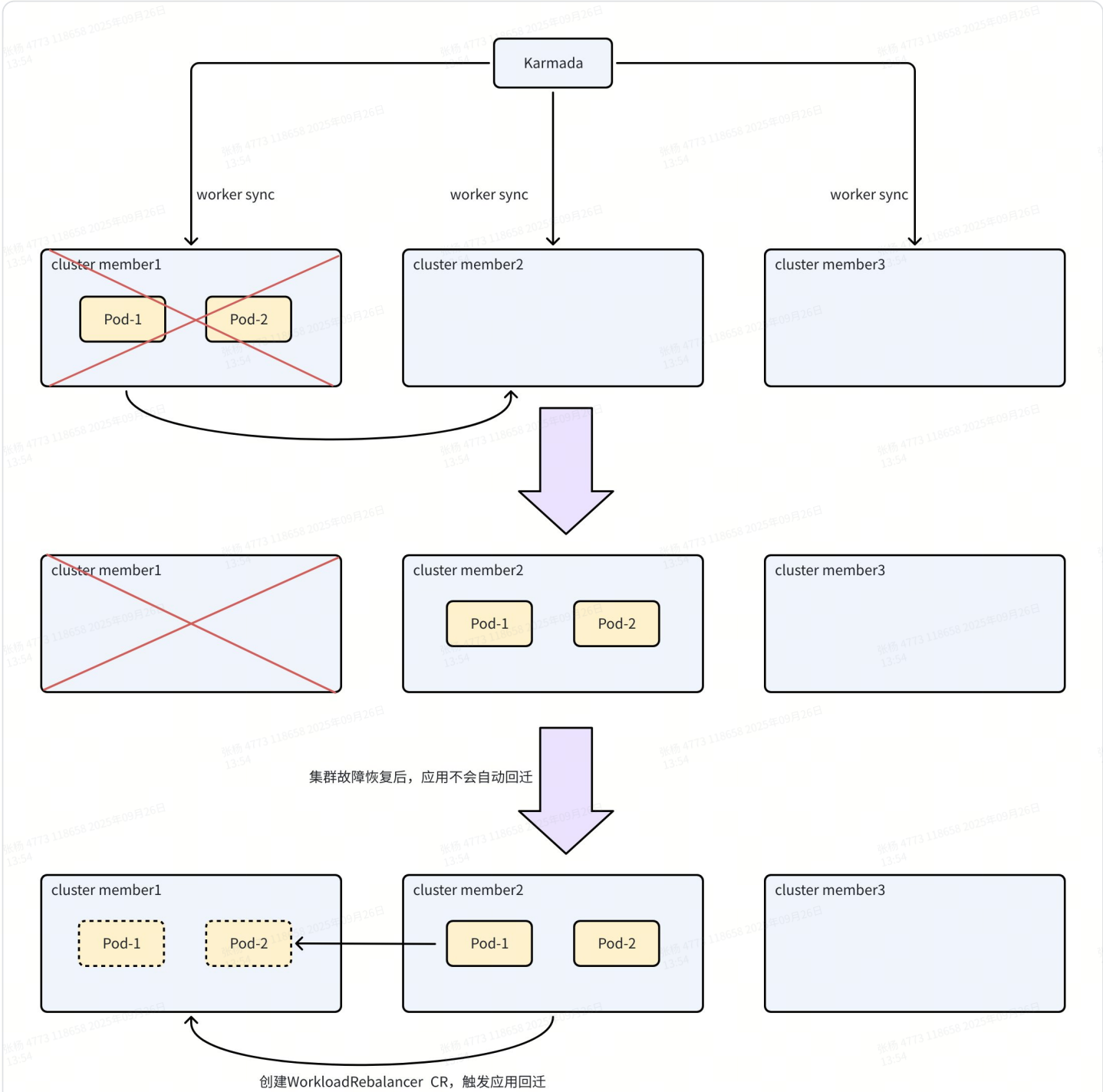
`PurgeMode` 代表如何从一个故障的集群迁移到另一个集群的方式。 Karmada 支持三种不同的驱逐模式：

- `Immediately` 表示 Karmada 将立即驱逐遗留的应用。
- `Graciously` 表示 Karmada 将等待应用在新集群上恢复健康，或者在达到超时后才驱逐应用。你同时需要配置 `GracePeriodSeconds`。如果新集群上的应用无法达到 Healthy 状态，Karmada 将在达到 GracePeriodSeconds 后删除该应用。默认为 600 秒。
- `Never` 表示 Karmada 不会驱逐应用，用户手动确认如何清理冗余副本。



4.6 应用重调度

一般情况下，工作负载类资源被调度后，会保持调度结果不变，其副本分布不会轻易发生变化。 现在，假设某些特殊情况下，想主动触发一次重调度，可以通过使用 WorkloadRebalancer 来实现。



代码块

```
$ kubectl --context karmada-apiserver get workloadrebalancer demo -o yaml
apiVersion: apps.karmada.io/v1alpha1
kind: WorkloadRebalancer
metadata:
  creationTimestamp: "2024-05-25T09:49:51Z"
  generation: 1
  name: demo
spec:
  workloads:
  - apiVersion: apps/v1
    kind: Deployment
```

```yaml
      name: foo
      namespace: default
  status:
    finishTime: "2024-05-25T09:49:51Z"
    observedGeneration: 1
    observedWorkloads:
    - result: Successful
      workload:
        apiVersion: apps/v1
        kind: Deployment
        name: foo
        namespace: default
```